

Boot 2 root

Summary: This document is an exercise in computer security.

Version: 6.0

Contents

I	**Preamble**	**2**
II	**Introduction**	**3**
III	**Objectives**	**4**
IV	**General instructions**	**5**
V	**Mandatory part**	**6**
VI	**Bonus part**	**8**
VII	**Submission and peer-evaluation**	**9**

Chapter I

Preamble



Chapter II

Introduction

Something is running on the 42 network. No instructions. No hints.

A server is up. Web application. Background services. Several user accounts with different privilege levels. Somewhere inside, a root shell.

Your job: get root. Twice, by two genuinely different routes. Then explain everything: not just what worked, but why it worked, what was broken, and how it could have been fixed.

The server hides **10 flags** along the path. Each flag is proof that a step was actually exploited, not skipped or guessed.

Forget spray-and-pray. Every credential, every open port, every file left behind is a deliberate choice. Read the system. Understand what you are looking at. That is what turns information into access.

Chapter III

Objectives

One server. One target. Get root.

Along the way you will practice reconnaissance, web exploitation, privilege escalation, binary analysis, cryptography, and filesystem forensics. Not as textbook exercises. As steps in a real attack chain, one leading to the next.

The server behaves like a real system. People left things behind. Services ran longer than they should have. Configuration choices were made that look fine on the surface. Nothing is random. Everything is there for a reason, but not every lead is worth following.

There are dead ends. Some paths look promising and go nowhere. Some details are planted to waste your time. Part of the challenge is knowing when to step back, re-read what you have, and find the thread that actually matters. Do not follow every rabbit hole.

When it is over, you will understand how real breaches happen: not through magic, but through the accumulation of small mistakes that nobody bothered to fix. You will know how to find them, chain them, and explain how each one could have been closed.



This project is deliberately complex. Spending a long time on a problem without making progress is part of the experience, not a sign of failure. Take your time, work together, and do not give up.



This project is done in a group. Reconnaissance, web exploitation, binary analysis, cryptography, system administration: different skills matter at different stages. Share everything you find. The picture only makes sense when all the pieces are on the table.

Chapter IV

General instructions

- This project will be reviewed by humans.
- You will have to prove your results during your evaluation. Be ready to walk through every step live.
- To complete this project, import the provided `.ova` appliance into VirtualBox and start it. Once the VM has finished booting, its console will look like this:



- From your host machine, the server is reachable on the network. Identifying its services is the first step of your reconnaissance.

Chapter V

Mandatory part

- The server hides **flags**. A flag is a unique token of the form `FLAG{...}` that you can only read once you have actually reached the privilege level that protects it. A recovered flag is your proof that the step was genuinely exploited.

- For the mandatory part you must:

  - perform a **reconnaissance** of the server and map its attack surface;

  - collect all **10 mandatory flags** distributed along the path to root. **2 of them** are won by exploiting the two distinct web entry methods, one flag per technique. The **remaining 8** are distributed along the privilege escalation chain from your initial foothold all the way to root;

  - reach a **root shell** (uid 0) via **at least 2 distinct web entry methods**. Each method represents a genuinely different web exploitation technique leading to an initial foothold on the server. A path that only differs in a detail of the same technique does not count as a second method;

  - produce a complete **write-up for each method**, describing every step taken, the vulnerability behind each step, and how it could have been prevented.

- Root means `uid=0` with a real interactive shell. Becoming root on a database or any other isolated service is not sufficient.

- Your repository layout must include:

```
$> ls
writeup1  writeup2  [scripts/]  [bonus/]
```

  The `scripts/` folder is optional. If present, it may contain scripts used during exploitation, but **no binaries**. The write-up must be written in English. Each step must be clearly described.



The root user means that the user id must be 0 and there must be a real shell where you can run commands such as whoami. Becoming root on another service is not a valid solution.



You cannot brute-force the server's login interfaces. Automated credential spraying against the running services will be rejected during evaluation. Offline cracking of captured material is a different matter and may be required.



We would like to point out that the **OVA must not be exploited directly**. You must **exploit the running SERVER** and not the file that runs it. Tricks exploiting the OVA file directly, exploiting the boot loader (GRUB), etc., are considered cheating.

- If you need a specific file from the server, download it during the evaluation. You must not include server files in your repository.
- If you use external tools, you must have set up the required environment (VM, Docker, Vagrant).
- You are invited to write scripts to speed up your work, but you must be able to explain them in detail during the evaluation.
- Explaining what you do is as important as the exploitation itself. Take the time to understand, and make sure you are understood.



WARNING: You must be able to fully explain everything in your repository. It must not contain **any binary**.

Chapter VI

Bonus part

Beyond the mandatory path, the server hides **5 additional flags**. Each flag is protected by a distinct vulnerability, completely independent of the main escalation chain.

For the bonus part, find and document as many of these **5 optional flags** as you can. For each one, hand in a write-up explaining the technique used to reach it, its impact, and how it could have been prevented. Each valid bonus earns one or two extra points (out of 5).

Take the time to explore the server **closely**: it would be a pity to rush this challenge.



The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been fully completed and works without any malfunction. If you have not met ALL the mandatory requirements, your bonus part will not be evaluated at all.



We would like to remind you once again that you must exploit the running SERVER, not the OVA file. Tricks exploiting the OVA file directly or the boot loader are considered cheating.

Chapter VII

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense.

How you organize your repository is **entirely up to you** as long as your work is easy to navigate and you can explain all of it during the evaluation. Don't hesitate to double-check the names of your folders and files to ensure they are correct.